

Diseño e implementación de un controlador difuso tipo Mamdani para la velocidad de un motor de corriente continua

Carlos Andrés Batista Figueroa — T00078355

Programa de Ingeniería Mecatrónica, Facultad de Ingeniería, Universidad Tecnológica de Bolívar

Asignatura: Control Inteligente — Docente: Ing. Ph.D. Efraín Rodríguez

Cartagena de Indias D. T. y C., Colombia — octubre de 2026

Abstract—This paper presents the design and experimental validation of a Mamdani fuzzy logic controller for the speed of a low-cost geared DC motor. The plant uses an Arduino Uno that only measures the speed with an FC-03 optical slot sensor and applies, through an L298N H-bridge, the PWM duty cycle it receives, while the controller runs in a Python desktop application that closes the loop at 40 Hz. Based on a literature review, a position-form fuzzy PI controller was designed with the speed error and the accumulated error as inputs, five triangular membership functions per input, six output sets, a 25-rule base, min-max inference and centroid defuzzification, plus conditional anti-windup and a feedforward-based preload. In simulation, the controller reached every setpoint within about 1 rpm and with slightly lower overshoot than a PID. On the real motor, it held 30 rpm and raised the applied power from 20.3 % to 32.4 % to reject a manual load disturbance.

Index Terms—Fuzzy control, Mamdani inference, membership functions, DC motor, speed control, Arduino.

I. INTRODUCCIÓN

Los sistemas de control están presentes en casi todos los procesos de la vida moderna, desde los electrodomésticos hasta las líneas de producción industrial, y su función es lograr que una variable física se comporte como se desea a pesar de las perturbaciones. Durante décadas, el diseño de estos sistemas se ha apoyado en modelos matemáticos de la planta y en controladores clásicos, como el proporcional-integral-derivativo, que ofrecen buenos resultados cuando el proceso es aproximadamente lineal y está bien caracterizado. Sin embargo, muchos sistemas reales presentan no linealidades, zonas muertas, saturaciones, parámetros inciertos o mediciones ruidosas, que dificultan obtener un modelo preciso y, con él, una sintonía adecuada. Frente a esas limitaciones surgió el control inteligente, un conjunto de técnicas que buscan reproducir en una máquina parte del razonamiento humano. Entre ellas, el control difuso ocupa un lugar destacado, porque permite expresar la estrategia de control mediante reglas formuladas en lenguaje cotidiano, como las que usaría un operador experimentado. Esa combinación de sencillez conceptual y capacidad para manejar la imprecisión explica su amplia adopción en la industria, en la electrónica de consumo y, en particular, en el accionamiento de máquinas eléctricas.

II. MARCO TEÓRICO

Durante buena parte del siglo XX, la ingeniería de control se apoyó en una premisa casi incuestionable: para gobernar un sistema primero había que describirlo con ecuaciones precisas. Sin embargo, las personas controlan a diario procesos complejos, como conducir un vehículo o regular el fuego de una estufa, sin resolver ninguna ecuación, guiadas por apreciaciones como “va muy rápido” o “falta un poco”. En 1965, Lotfi A. Zadeh, profesor de la Universidad de California en Berkeley, propuso una herramienta matemática para representar ese conocimiento impreciso: los conjuntos difusos (Zadeh, 1965). A diferencia de la teoría clásica, en la que un elemento pertenece o no a un conjunto, en un conjunto difuso la pertenencia es gradual y se expresa con un número entre 0 y 1. Así, un motor que gira a 300 rpm puede ser “rápido” en grado 0,6 y “medio” en grado 0,4 al mismo tiempo.

Pocos años después, Zadeh (1973) planteó que los sistemas complejos se describen mejor con reglas formuladas en palabras que con modelos numéricos exactos, y para ello formalizó el concepto de *variable lingüística* (Zadeh, 1975): una variable cuyos valores no son números, sino términos del lenguaje natural. El error de velocidad, por ejemplo, puede tomar los valores “negativo grande”, “cero” o “positivo pequeño”, y cada término queda definido por un conjunto difuso sobre el *universo de discurso*, es decir, sobre el rango numérico que puede tomar la variable.

La forma de cada conjunto la fija su *función de membresía*, que asigna a cada valor del universo su grado de pertenencia. Las más usadas son la triangular, la trapezoidal y la gaussiana; la triangular es la preferida en control porque es sencilla de calcular y, con pocos parámetros, representa bien la información (Pedrycz, 1994). Lo habitual es que los conjuntos vecinos se solapen para que el paso de un término a otro sea suave, y cuando los grados de pertenencia de cualquier valor suman uno, la familia de conjuntos forma una partición de Ruspini (Ruspini, 1969).

La idea de Zadeh llegó al control de la mano de Ebrahim Mamdani, quien, junto con Sedrak Assilian, gobernó en el Queen Mary College de Londres una pequeña máquina de vapor con reglas que imitaban la estrategia de un operador humano, y demostró que el método funcionaba en una planta

real (Mamdani, 1974; Mamdani y Assilian, 1975). Ese éxito abrió paso a aplicaciones industriales como el control de hornos de cemento en Dinamarca (Holmblad y Østergaard, 1982) y la operación automática del metro de Sendai, en Japón (Yasunobu y Miyamoto, 1985).

El conocimiento del experto se guarda en la *base de reglas*, formada por proposiciones del tipo SI-ENTONCES. Cada regla une un antecedente, que describe una situación (“si el error es positivo grande”), con un consecuente, que indica la acción (“entonces la potencia es alta”). Con dos entradas, las reglas suelen organizarse en una tabla que cubre todas las combinaciones de sus términos (Lee, 1990a).

Con estos elementos, un controlador difuso trabaja en cuatro etapas (Lee, 1990a; Passino y Yurkovich, 1998). En la *fuzzificación*, cada medición se convierte en grados de pertenencia a los términos de su variable. En la *inferencia* se evalúan las reglas: en el método de Mamdani, la fuerza de cada regla es el mínimo de los grados de sus antecedentes, esa fuerza recorta el conjunto de su consecuente, y la unión de todos los recortes, tomada con el operador máximo, forma un único conjunto difuso de salida (Lee, 1990b). Takagi y Sugeno (1985) propusieron una alternativa en la que el consecuente es una función de las entradas. Por último, la *defuzzificación* convierte el conjunto de salida en el número que se envía al actuador; entre métodos como la bisectriz o la media de los máximos, el más empleado es el centroide, que toma el centro de gravedad del área y produce una acción suave (Van Leekwijck y Kerre, 1999). El resultado es una superficie de control no lineal que interpola entre las reglas y que se construye a partir del conocimiento del problema, sin necesidad de un modelo exacto de la planta (Ross, 2010).

III. REVISIÓN DE LITERATURA

El control difuso de la velocidad de motores de corriente continua es uno de los casos de estudio más frecuentes en la literatura, tanto en simulación como en plataformas reales de bajo costo. Usoro et al. (2017) compararon en MATLAB un controlador Mamdani con un PID para un motor de tipo industrial, y Ridzuan y Abdul Rahman (2024) llevaron esa comparación a tiempo real con un microcontrolador STM32. El trabajo más cercano al presente es el de Petrescu et al. (2026), que regularon motorreductores 1:48 de bajo costo con un Arduino Uno, un puente L298 y un encoder óptico de 20 ranuras. Esos autores señalaron como principales dificultades la zona muerta, la fricción, la baja resolución del encoder y las variaciones de tensión de la batería.

En cuanto a la estructura, se distinguen dos enfoques. En el directo, el controlador difuso calcula la acción de control (Usoro et al., 2017; Petrescu et al., 2026); en el híbrido, las reglas ajustan en línea las ganancias de un PID convencional (Zhao et al., 1993). Casi todos los trabajos usan como entradas el error y su cambio (Lee, 1990a). Cuando la salida es absoluta, esa pareja da un controlador de tipo PD, sin acción integral. Para obtenerla, Petrescu et al. (2026) hicieron que la salida fuera un incremento de PWM que se acumula en cada muestra, mientras que otros autores combinan el controlador

difuso con un término integral (Qiao y Mizumoto, 1996; Mann et al., 1999).

Las funciones de membresía triangulares son las más empleadas, por su bajo costo de cálculo en sistemas embebidos, aunque también se usan trapezoidales (Petrescu et al., 2026). Las bases de reglas suelen tener cinco términos por entrada, es decir, 25 reglas (Ridzuan y Abdul Rahman, 2024), o siete, con 49 reglas (Usoro et al., 2017; Petrescu et al., 2026), y casi siempre usan inferencia de Mamdani min-max con defuzzificación por centroide. La inferencia de Takagi y Sugeno (1985) aparece como alternativa. Un hallazgo útil para el diseño es que los conjuntos trapezoidales, con una meseta en error cero, dejaron un error permanente que no apareció con triángulos (Petrescu et al., 2026).

El desempeño se evalúa sobre todo con la respuesta al escalón: tiempo de subida, sobreimpulso, tiempo de establecimiento y error en estado estacionario. Ridzuan y Abdul Rahman (2024), por ejemplo, reportaron un sobreimpulso de 0,77 % con el controlador difuso frente a 3,46 % con el PID. Varios trabajos añaden pruebas con par de carga (Usoro et al., 2017) e índices integrales como IAE, ITAE y RMSE. Petrescu et al. (2026) incluyeron además la variación total de la señal de control: el PID obtuvo menores índices de error y una subida más rápida, pero el difuso redujo esa variación en un 56 % y dio una respuesta más suave. En conjunto, la literatura muestra que el control difuso es especialmente útil en plantas no lineales con mediciones pobres. Sin embargo, ninguno de los trabajos de aplicación revisados usa el error acumulado como entrada de un controlador de salida absoluta, que es el enfoque adoptado en este trabajo.

IV. DESCRIPCIÓN DEL SISTEMA ESTUDIADO

La planta es la misma plataforma del trabajo previo de control PID: un motorreductor de corriente continua tipo TT, con reducción 1:48 y tensión nominal de 3 a 6 V (Figs. 1 y 2). Dos baterías de litio 18650 en serie alimentan un convertidor reductor LM2596, que ajusta la tensión de trabajo del puente H L298N. El Arduino Uno entrega al puente, por el pin D3, una señal PWM de unos 490 Hz, cuyo ciclo útil fija la tensión media en bornes del motor, y por D7 y D8 las señales IN1 e IN2, que definen el sentido de giro. La velocidad se mide con un sensor óptico de herradura FC-03 (comparador LM393) que lee un disco de 20 ranuras montado en el eje de salida; su salida llega al pin D2, asociado a la interrupción externa INT0. Todas las tierras están unidas. Un condensador cerámico de 100 nF entre los bornes del motor atenúa el ruido de las escobillas, y uno electrolítico de 1 μ F desacopla la alimentación del sensor.

El firmware del Arduino es el mismo del trabajo anterior y solo mide la velocidad y aplica el PWM que recibe. Cada flanco de bajada del sensor activa una rutina de interrupción con dos filtros. El primero espera 8 μ s y descarta el flanco si la línea ya volvió a nivel alto, porque los picos inducidos por la conmutación del PWM duran pocos microsegundos, mientras que una ranura real mantiene la señal en bajo durante cientos de microsegundos. El segundo descarta los flancos que llegan

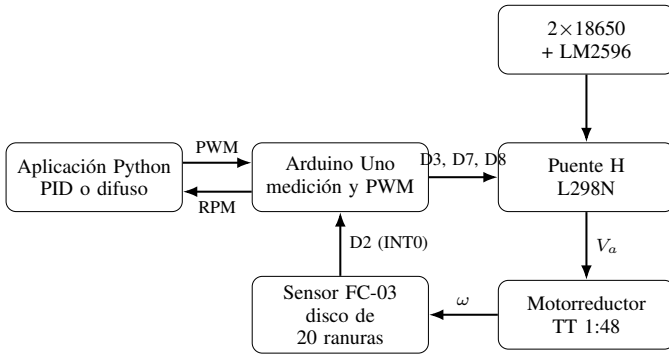


Fig. 1. Diagrama general del sistema: el controlador corre en el computador y el Arduino solo mide la velocidad y aplica el PWM.

a menos de $250 \mu s$ del anterior, producidos por rebotes del comparador. Cada 25 ms se calcula la velocidad de flanco a flanco:

$$RPM = \frac{60 \times 10^6 \cdot N_{\text{pulsos}}}{\Delta t_{\mu s} \cdot N_{\text{ranuras}}} \quad (1)$$

donde $\Delta t_{\mu s}$ es el tiempo exacto entre el último flanco de la ventana anterior y el de la actual, lo que evita el error de cuantización de contar pulsos en una ventana fija. El resultado pasa por una mediana de las cinco últimas medidas, y si transcurren 300 ms sin pulsos el motor se da por detenido. La placa envía a 9600 baudios la velocidad y el PWM aplicado, y solo acepta tres órdenes: el PWM con signo, la parada y el número de ranuras. El controlador, PID o difuso, se calcula en la aplicación de escritorio en Python, que con cada medida envía el nuevo PWM, de modo que el lazo se cierra a 40 Hz a través del puerto serie.

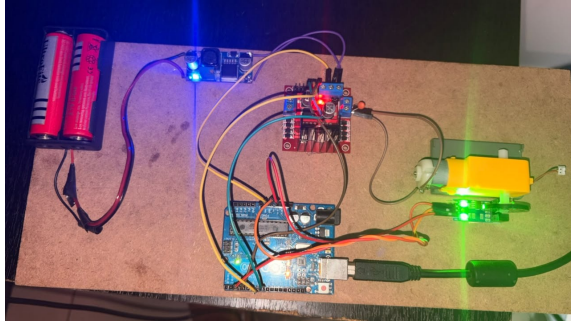


Fig. 2. Montaje físico: baterías 18650, convertidor LM2596, puente H L298N, Arduino Uno, motorreductor y sensor FC-03 con el disco ranurado.

V. DISEÑO DEL CONTROLADOR DIFUSO

V-A. Estructura y variables de entrada

El controlador calcula directamente el ciclo útil del PWM, es decir, una potencia absoluta entre 0 y 100 %. Con la pareja clásica de error y cambio del error (Lee, 1990a), en régimen ($e = 0$ y $\Delta e = 0$) el controlador entregaría siempre la misma potencia, y el motor solo alcanzaría la consigna que corresponde a ese valor. Por eso se eligieron como entradas el error y el error acumulado, lo que da un PI difuso en forma de posición (Mann et al., 1999): el acumulado es la memoria

que encuentra la potencia que necesita cada setpoint. Con cada medida, que llega cada $T_s = 25$ ms, se calculan

$$e_k = |SP| - \text{rpm}_k, \quad \int e_k = \int e_{k-1} + e_k T_s \quad (2)$$

El sensor no distingue el sentido de giro, así que se regula la magnitud de la velocidad, y el signo del setpoint fija el sentido mediante IN1 e IN2. El acumulado se limita a $\pm \int e_{\text{máx}}$, y las dos entradas se normalizan y se recortan a $[-1, 1]$:

$$e_n = \text{sat}_{[-1,1]} \left(\frac{e}{e_{\text{máx}}} \right), \quad a_n = \text{sat}_{[-1,1]} \left(\frac{\int e}{\int e_{\text{máx}}} \right) \quad (3)$$

Por defecto, $e_{\text{máx}} = 200$ rpm y $\int e_{\text{máx}} = 100$ rpm·s, ambos ajustables en caliente; un rango menor hace al controlador más enérgico.

V-B. Variables lingüísticas y funciones de membresía

Las entradas comparten cinco términos: negativo grande (NG), negativo pequeño (NP), cero (Z), positivo pequeño (PP) y positivo grande (PG). La salida tiene seis: potencia cero (PC), baja (PB), media baja (PMB), media (PM), media alta (PMA) y alta (PA). Todos los conjuntos son triangulares, por su bajo costo de cálculo (Pedrycz, 1994):

$$\mu(x) = \max \left(0, 1 - \frac{|x - c|}{b} \right) \quad (4)$$

donde c es el centro y b la semibase. En las entradas, los triángulos están centrados en $-1, -0,5, 0, 0,5$ y 1 , con $b = 0,5$ (Fig. 3, arriba). Cada uno se solapa a la mitad con sus vecinos, de modo que un valor pertenece como máximo a dos conjuntos y sus grados suman uno, lo que forma una partición de Ruspini (Ruspini, 1969). En la salida, los triángulos tienen $b = 20$ % y están centrados de 0 a 100 % cada 20 % (Fig. 3, abajo). Los de los extremos se dejaron completos para que el centroide pueda llegar a 0 y a 100 %. El universo de salida se discretiza en 561 puntos entre -20 y 120 %.

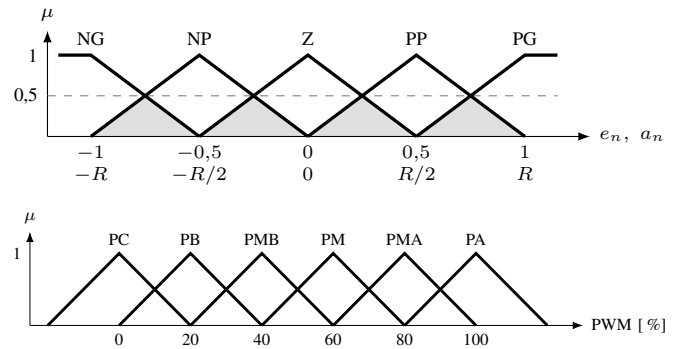


Fig. 3. Funciones de membresía. Arriba: entradas normalizadas, con $R = e_{\text{máx}}$ para el error y $R = \int e_{\text{máx}}$ para el error acumulado; las zonas sombreadas son las áreas de solapamiento. Abajo: salida.

V-C. Base de reglas

Las 25 reglas tienen la forma “SI e es A Y $\int e$ es B ENTONCES la potencia es C” y se numeran de R1 a R25 recorriendo por filas la Tabla I. Cerca de $e = Z$, cada paso

de una entrada sube o baja un conjunto de salida, lo que da una respuesta suave alrededor de la consigna. En las filas NG y PG, en cambio, el salto es de tres conjuntos, de modo que lejos de la consigna la acción es enérgica. Esa no linealidad distingue al controlador de un PI lineal. La fila Z va de PB a PA, así que con error nulo el controlador puede sostener potencias entre 20 y 100 %; por debajo del 20 % el motor estaría junto a su zona muerta, medida en 15 %.

Tabla I
BASE DE REGLAS DIFUSAS (CONSECUENTE: POTENCIA DE SALIDA)

$e / \int e$	NG	NP	Z	PP	PG
NG	PC	PC	PC	PB	PMB
NP	PC	PB	PMB	PM	PMA
Z	PB	PMB	PM	PMA	PA
PP	PMB	PM	PMA	PA	PA
PG	PMA	PA	PA	PA	PA

V-D. Inferencia y defuzzificación

Se usó el método de Mamdani (Mamdani y Assilian, 1975). La fuerza de cada regla es el mínimo de sus antecedentes, esa fuerza recorta su consecuente y los recortes se unen con el operador máximo. El centroide convierte el conjunto agregado en el PWM que se envía a la placa (Van Leekwijck y Kerre, 1999):

$$w_{ij} = \min(\mu_i(e_n), \mu_j(a_n)), \quad \mu_{ag}(x) = \max_{ij} \min(w_{ij}, \mu_{R(i,j)}(x)) \quad (5)$$

$$u^* = \frac{\sum_x \mu_{ag}(x) x}{\sum_x \mu_{ag}(x)}, \quad \text{PWM} = \text{sat}_{[0, \text{PWM}_{\max}]}(u^*) \quad (6)$$

En cada muestra disparan como máximo cuatro reglas. Cerca del origen, cada media unidad normalizada de entrada mueve la salida un conjunto (20 %), así que el controlador equivale localmente a un PI con

$$K_{p,eq} \approx \frac{40}{e_{\max}} \frac{\%}{\text{rpm}}, \quad K_{i,eq} \approx \frac{40}{\int e_{\max}} \frac{\%}{\text{rpm} \cdot \text{s}} \quad (7)$$

es decir, 0,2 %/rpm y 0,4 %/(rpm·s) con los valores por defecto. Son del orden del PID del trabajo anterior, que equivalía a 0,17 %/rpm y 0,5 %/(rpm·s), así que los dos controladores parten de una agresividad comparable.

V-E. Anti-windup, precarga e implementación

Además del límite del acumulado, se aplica integración condicional: el error no se acumula si $|e| \geq e_{\max}$ o si la salida ya está en su tope. Para que un escalón no tenga que esperar a que el acumulado viaje desde cero, al aplicar un setpoint el acumulado se precarga con el valor que, con error nulo, da el PWM de la prealimentación

$$ff = zm + |SP| \cdot \frac{100 - zm}{\text{rpm}_{\max}} \quad (8)$$

con $zm = 15\%$ y $\text{rpm}_{\max} = 600$. Como la salida de la fila Z crece de forma monótona con el acumulado, ese valor se halla por bisección. La precarga solo fija el estado inicial, y después el acumulado corrige la diferencia entre el modelo y el motor

real. El controlador se programó en Python con numpy, sin bibliotecas de lógica difusa, y cada cálculo tarda en promedio 0,046 ms, muy por debajo del periodo de 25 ms.

VI. RESULTADOS DE SIMULACIÓN Y ANÁLISIS

VI-A. Interfaz del controlador difuso

La Fig. 4 muestra la vista de control difuso de la aplicación, que sigue el orden del cálculo. Arriba aparecen la respuesta del motor con el PWM aplicado, la fuzzificación de las dos entradas con sus grados de pertenencia, la base de reglas con las reglas activas iluminadas según su fuerza, la inferencia y la agregación con el centroide. Abajo aparecen la superficie de control $u^*(e, \int e)$, calculada en 61×61 puntos, el estado del lazo, el monitor serie y los parámetros del controlador.

VI-B. Verificación en simulación

Con error nulo, la potencia del controlador aislado crece de forma monótona de 20 a 100 %, y la superficie precalculada coincide con la inferencia directa. Después se cerró el lazo contra un modelo de primer orden del motor, muestreado cada 25 ms (600 rpm con PWM al 100 %, zona muerta del 15 % y constante de tiempo supuesta de 0,25 s), con consignas de 300, 100 y 450 rpm (Tabla II). El difuso terminó cada tramo a cerca de 1 rpm de la consigna, con un pico de 317 rpm en el primer escalón (5,7 % de sobreimpulso), frente a 322 rpm (7,3 %) del PID. Con un motor un 15 % más débil que el modelo de la precarga, el acumulado corrigió la diferencia y el sobreimpulso bajó a 0,7 %.

Tabla II
VELOCIDAD FINAL DE CADA TRAMO Y PICO DEL PRIMER ESCALÓN, EN RPM (SIMULACIÓN)

Caso	300	100	450	Pico
Difuso	298,9	98,9	450,2	317
PID	298,5	100,0	450,0	322
Difuso, motor 15 % más débil	298,7	99,5	450,2	302

VI-C. Ensayos con el motor real

Con el motor real se aplicó una consigna de 30 rpm con los valores por defecto (Fig. 5a). Al arrancar, el PWM sube por encima del 25 % hasta vencer la fricción estática. Cuando el motor se suelta, la velocidad sobrepasa la consigna hasta unas 55 rpm y se asienta en torno a 30 rpm cerca de un segundo después. En régimen, el error es de $-0,3$ rpm (Z) y el acumulado de $-99,4$ rpm·s (NG 0,99), así que dispara casi sola la regla R11, “si e es Z y $\int e$ es NG, entonces la potencia es baja”, y el centroide da 20,3 %.

En el segundo ensayo se frenó el eje con el dedo una vez asentada la respuesta (Fig. 5b). La velocidad cayó hasta unas 20 rpm, el acumulado empezó a crecer y la potencia subió de forma gradual hasta devolver la velocidad a la consigna sin retirar la carga. Con carga, el acumulado llegó a $-69,3$ rpm·s y dispararon R12 (PMB, $w = 0,61$) y R11 (PB, $w = 0,39$). El conjunto agregado tomó entonces una forma escalonada, con el centroide en 32,4 % (Tabla III). El controlador aplicó así 12 puntos más de PWM para mantener las 30 rpm, sin cambiar ninguno de sus parámetros.

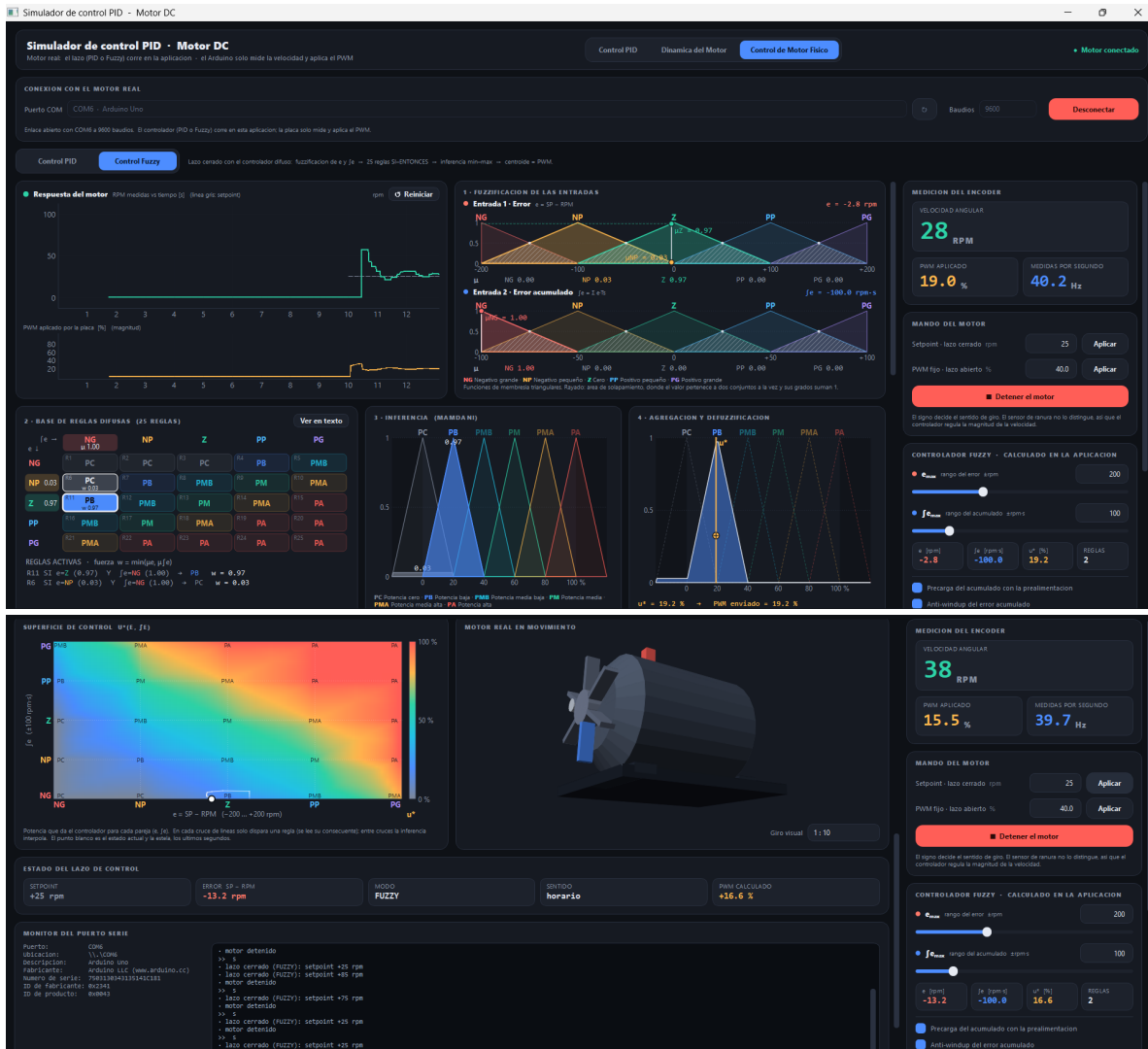


Fig. 4. Vista de control difuso de la aplicación conectada al motor real. Arriba: respuesta del motor y PWM aplicado, fuzzificación de las entradas, base de reglas con las reglas activas, inferencia y agregación con el centroide. Abajo: superficie de control, motor 3D, estado del lazo, monitor serie y parámetros del controlador.

Tabla III
ESTADO DEL CONTROLADOR EN RÉGIMEN A 30 RPM

	Sin carga	Con carga
e [rpm]	-0,3 (Z 1,00)	+2,0 (Z 0,98)
$\int e$ [rpm·s]	-99,4 (NG 0,99)	-69,3 (NG 0,39; NP 0,61)
Reglas	R11 $\rightarrow$ PB (0,99)	R12 $\rightarrow$ PMB (0,61); R11 $\rightarrow$ PB (0,39)
u^* (PWM)	20,3 %	32,4 %

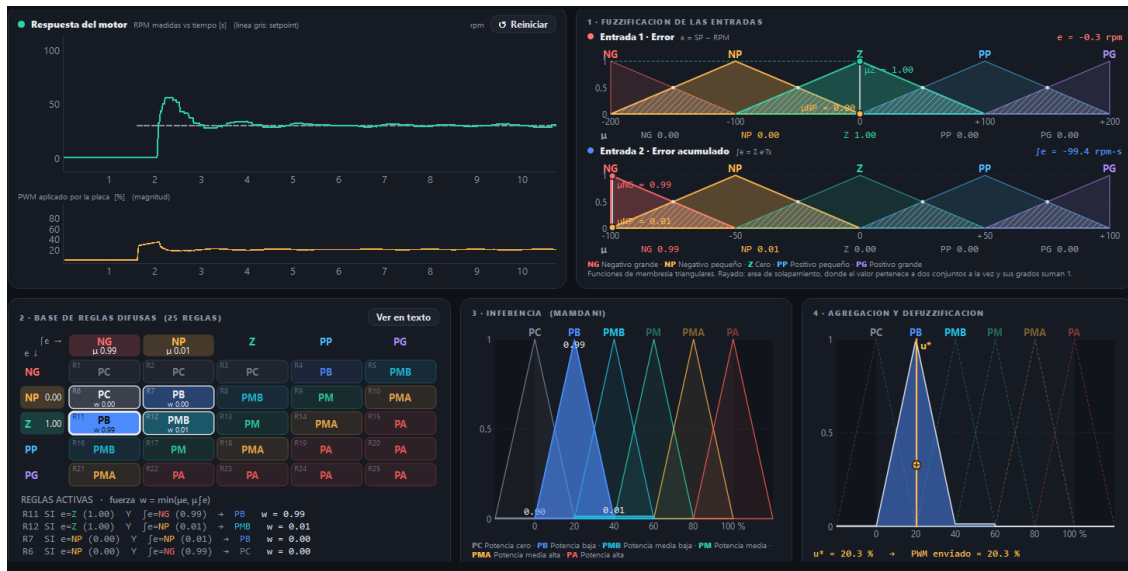
Este resultado coincide con el diseño. El acumulado cambió 30,1 rpm·s y, con $K_{i,eq} \approx 0,4 \text{ \%/rpm·s}$ según la Ec. (7), eso equivale a 12,0 % de PWM, frente a los 12,1 % medidos. El aporte del error presente es pequeño (cerca de 0,4 %), así que la carga la rechaza la memoria del controlador, y por eso la recuperación tarda unos segundos; reducir $\int e_{\text{máx}}$ la aceleraría, a costa de más sobreimpulso. El sobreimpulso al arrancar, de unas 25 rpm, es mucho mayor que en simulación porque la fricción estática exige un 19 % de PWM para arrancar pero

solo un 15 % para mantener el giro, y el modelo no incluye ese efecto. En todo caso, dura menos de medio segundo.

VI-D. Comportamiento a muy baja velocidad

Con consignas de 25 rpm aparece una limitación prevista en el diseño. La fila Z no baja de 20 % y, como el motor gira incluso con 15 %, con 20 % va más rápido que la consigna. Para reducir la potencia, el controlador necesita un error negativo, así que la velocidad se estabiliza algo por encima de la consigna con el acumulado saturado en -100 rpm·s . En la Fig. 4 se observan estados con 28 rpm y $u^* = 19,2 \%$, y con 38 rpm, $e = -13,2 \text{ rpm}$ y $u^* = 16,6 \%$, con el estado actual sobre el borde inferior de la superficie. Además, a 25 rpm llega un pulso cada 120 ms, casi cinco periodos de muestreo, y la lectura avanza a saltos. Bajar el consecuente de la regla R11 a PC corregiría la limitación.

Enlace al video de los ensayos: <https://drive.google.com/file/d/1IoQRMT2YNDWYaHPtG9Ic619yaPzhIRUC/view?usp=>



(a) Sin carga



(b) Con carga aplicada con el dedo

Fig. 5. Ensayos con el motor real a 30 rpm. (a) En régimen dispara casi sola la regla R11 (PB) y el centroide queda en 20,3 %. (b) Con carga, el acumulado pasa a NG 0,39 y NP 0,61, disparan R12 (PMB) y R11 (PB), y el centroide sube a 32,4 %.

sharing

Enlace al proyecto completo:
<https://github.com/CarlosBatista08/SIMULATOR-CONTROLLER-PID-FUZZY-FOR-DC-MOTOR>

VII. CONCLUSIONES

El controlador difuso diseñado reguló la velocidad del motor de corriente continua tanto en simulación como con el motor real. Su estrategia quedó escrita en reglas fáciles de leer, lo que permitió relacionar cada decisión de diseño con el comportamiento observado.

La elección de las entradas fue determinante. Como la salida es una potencia absoluta, usar el error y el error acumulado convirtió al controlador en un PI difuso que elimina el error en estado estacionario, algo que no habría logrado la pareja

clásica de error y cambio del error. Además, la base de reglas dio una acción enérgica lejos de la consigna y suave cerca de ella.

En simulación, el controlador alcanzó prácticamente las mismas velocidades finales que el PID, con un sobreimpulso algo menor, y mantuvo su desempeño con un motor un 15 % más débil. Con el motor real sostuvo las 30 rpm y, ante una carga aplicada con el dedo, elevó la potencia de 20,3 a 32,4 %, un aumento que coincide con la ganancia integral equivalente prevista en el diseño. Las principales limitaciones fueron la potencia mínima de la fila Z, que impide sostener sin error consignas por debajo de unas 30 rpm, y la baja resolución del encoder de 20 ranuras, que sigue siendo el factor que más restringe la calidad del control.

BIBLIOGRAFÍA

- Holmblad, L. P., y Østergaard, J. J. (1982). Control of a cement kiln by fuzzy logic. En M. M. Gupta y E. Sanchez (Eds.), *Fuzzy information and decision processes* (pp. 389–399). North-Holland.
- Lee, C. C. (1990a). Fuzzy logic in control systems: Fuzzy logic controller, Part I. *IEEE Transactions on Systems, Man, and Cybernetics*, 20(2), 404–418.
- Lee, C. C. (1990b). Fuzzy logic in control systems: Fuzzy logic controller, Part II. *IEEE Transactions on Systems, Man, and Cybernetics*, 20(2), 419–435.
- Mamdani, E. H. (1974). Application of fuzzy algorithms for control of simple dynamic plant. *Proceedings of the Institution of Electrical Engineers*, 121(12), 1585–1588.
- Mamdani, E. H., y Assilian, S. (1975). An experiment in linguistic synthesis with a fuzzy logic controller. *International Journal of Man-Machine Studies*, 7(1), 1–13.
- Mann, G. K. I., Hu, B.-G., y Gosine, R. G. (1999). Analysis of direct action fuzzy PID controller structures. *IEEE Transactions on Systems, Man, and Cybernetics, Part B: Cybernetics*, 29(3), 371–388.
- Passino, K. M., y Yurkovich, S. (1998). *Fuzzy control*. Addison-Wesley.
- Pedrycz, W. (1994). Why triangular membership functions? *Fuzzy Sets and Systems*, 64(1), 21–30.
- Petrescu, I., Băjenaru, V.-D., Popescu, D.-M., Vulturescu, V., y Ungureanu, L. M. (2026). A comparative study of PID and bio-inspired fuzzy controllers for speed regulation of low-cost geared DC motors. *Biomimetics*, 11(9), 643. <https://doi.org/10.3390/biomimetics11090643>
- Qiao, W. Z., y Mizumoto, M. (1996). PID type fuzzy controller and parameters adaptive method. *Fuzzy Sets and Systems*, 78(1), 23–35.
- Ridzuan, A., y Abdul Rahman, H. (2024). Real time comparison between PID and fuzzy logic controller for DC motor speed control. *Evolution in Electrical and Electronic Engineering*, 5(1), 512–520.
- Ross, T. J. (2010). *Fuzzy logic with engineering applications* (3.^a ed.). John Wiley & Sons.
- Ruspini, E. H. (1969). A new approach to clustering. *Information and Control*, 15(1), 22–32.
- Takagi, T., y Sugeno, M. (1985). Fuzzy identification of systems and its applications to modeling and control. *IEEE Transactions on Systems, Man, and Cybernetics, SMC-15*(1), 116–132.
- Uoro, I. H., Itaketo, U. T., y Umoren, M. A. (2017). Control of a DC motor using fuzzy logic control algorithm. *Nigerian Journal of Technology*, 36(2), 594–602. <https://doi.org/10.4314/njt.362.1313>
- Van Leekwijck, W., y Kerre, E. E. (1999). Defuzzification: Criteria and classification. *Fuzzy Sets and Systems*, 108(2), 159–178.
- Yasunobu, S., y Miyamoto, S. (1985). Automatic train operation system by predictive fuzzy control. En M. Sugeno (Ed.), *Industrial applications of fuzzy control* (pp. 1–18). North-Holland.
- Zadeh, L. A. (1965). Fuzzy sets. *Information and Control*, 8(3), 338–353.
- Zadeh, L. A. (1973). Outline of a new approach to the analysis of complex systems and decision processes. *IEEE Transactions on Systems, Man, and Cybernetics, SMC-3*(1), 28–44.
- Zadeh, L. A. (1975). The concept of a linguistic variable and its application to approximate reasoning—I. *Information Sciences*, 8(3), 199–249.
- Zhao, Z.-Y., Tomizuka, M., y Isaka, S. (1993). Fuzzy gain scheduling of PID controllers. *IEEE Transactions on Systems, Man, and Cybernetics*, 23(5), 1392–1398.